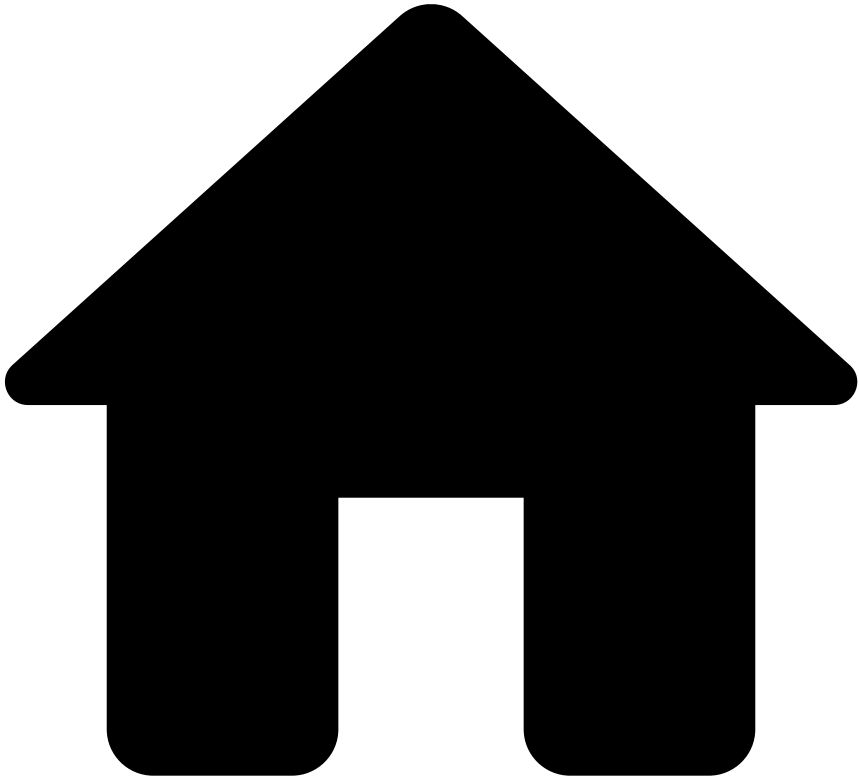


•

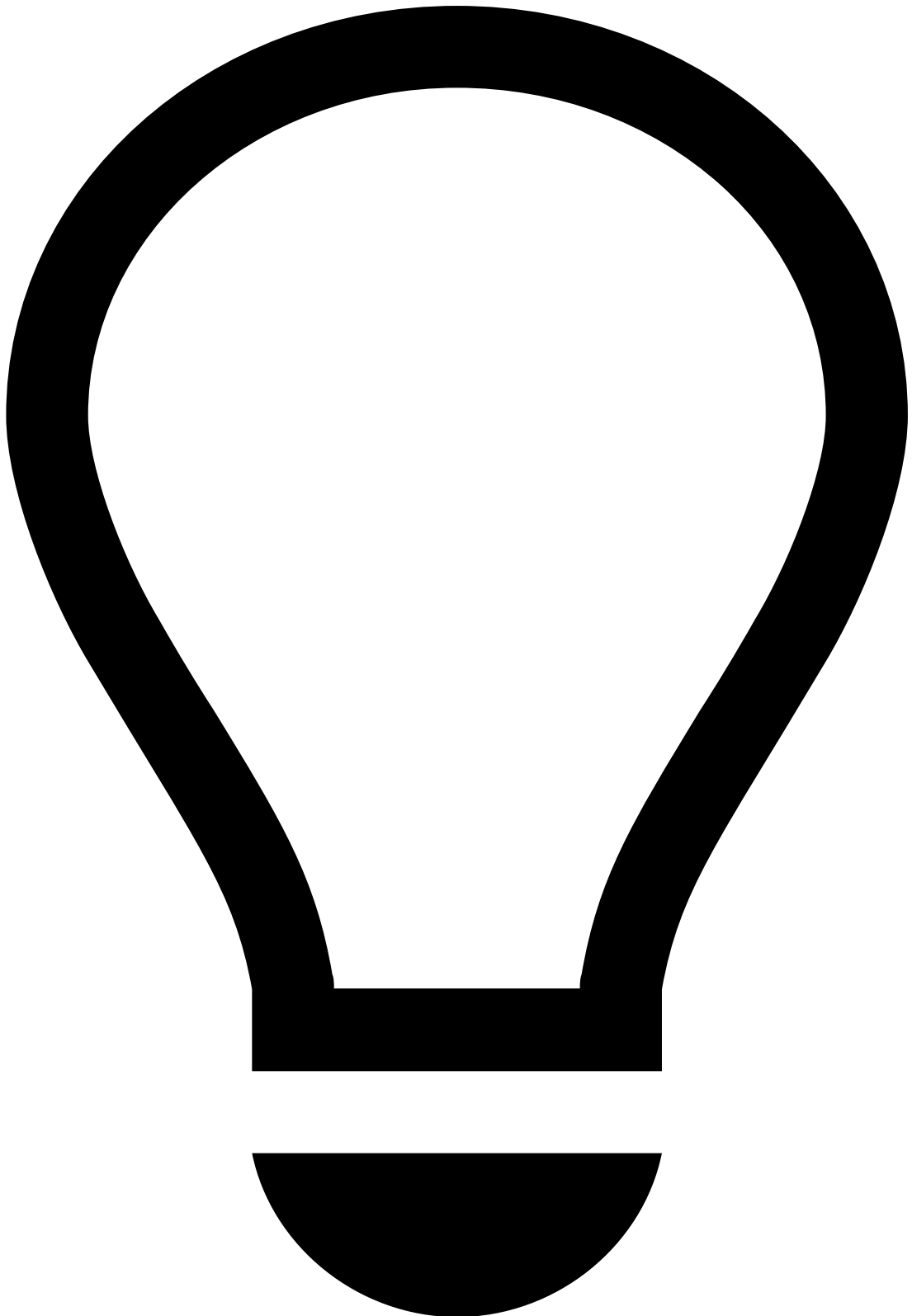


- [Core Concepts](#)
- [Workflows](#)
- Scheduled Metrics Jobs

On this page

Scheduled Metrics Jobs

Was this page helpful? 



Page content not applicable to Pulse with Railgun

When Pulse is configured with the [Railgun engine](#), it is able to calculate all metrics in real time. Therefore, the concept of scheduled metrics, as described in this page, is not applicable.

Scheduled Metrics Jobs (SMJs) are responsible for computing the non-real time metrics (i.e., [scheduled metrics](#)) of profiles at periodic times in batches and persisting them in the profile store, thus making the calculated results available to scoring.

SMJs are a particular type of job , and a specific type of workflow job, that gets automatically registered by Pulse, for example when a [Plan](#) that calculates scheduled metrics is deployed in a runtime [Workflow](#).

Any service in a workflow has the capability to execute jobs and implements them differently. A workflow job can be declared to run periodically, according to the [workflow clock](#), and/or on-demand. For example, if you write a custom service that updates the list of blacklisted emails in an external service, you can declare it to register a workflow job that runs every week.

In the case of plan Services in a workflow, it will register SMJs if there are scheduled metrics configured in the underlying Data Science [Plan](#).

Registered Jobs

An SMJ is registered for each unique metric schedule in a metrics [Plan](#). When the SMJ is executed, Pulse calculates all metrics corresponding to its schedule.

SMJs are launched automatically by the Workflow Job Manager scheduler as events are processed when the workflow clock reaches the SMJ schedule. As with the registered jobs, the SMJs that are launched by the scheduler are those for all metrics with the same schedule in each metrics plan.

Registered jobs are available in the Pulse UI (**Jobs** tab), where the user can launch them on-demand whenever he/she wishes (e.g., recalculate metrics in case of errors).

Regardless of being launched by the scheduler or on-demand, SMJs can run internally (within some AppEngine) or externally, in separate Java processes, making them isolated from each other and from the App Engine. The Job Manager ensures there are no more than a specified number of concurrent SMJs running. Should additional ones be scheduled for execution, they will wait until running jobs finish.

Job Execution

Each SMJ execution processes all metrics corresponding to its schedule, with a single pass in [Event Storage](#) for a period with a duration equal to the maximum window time span among all processed metrics.

In case of Omnichannel plans ([Plans- Deployable Omnichannel Plans](#)), the job will potentially access multiple Event Storages – the one for the main channel, as well as the ones for the secondary channels which are used in the scheduled metrics. Events across all Event Storages will be processed in chronological order (as opposed to processing the Event Storages one by one).

The period end time is determined by how the SMJ is launched:

- If launched by the scheduler, it is the scheduled time.
- If launched on demand, it is the current workflow clock, i.e., the timestamp of the last event processed.

Note that the scheduler fires SMJ executions based on the workflow clock, not on the system time. Thus, if no events are processed, no scheduled SMJs will be launched. See [Time, schedules and timezones](#) for more details on this.

Partitioned Scheduled Metrics Jobs

Processing all events in a single pass may require a significant amount of memory when the metric grouping columns are of high cardinality. You can reduce memory consumption by partitioning the data

and at the same time [configuring SMJs](#) to read the Event Storage in a partitioned way, processing only a part of it on each core.

All SMJ configurations are specified per **Plan Service** element of each workflow. Therefore, SMJs are partitioned only if partitioning is explicitly enabled in the workflow. If not, SMJs will run in normal mode.

Registered Jobs

A metrics [plan](#) is partitionable if and only if all its metrics are partitionable. A metric is partitionable if and only if all workflow partitions include all partition columns. For example:

A workflow is partitioned by:

- `account_id` and

- *merchant_id*

A plan service executes a plan with:

- A daily scheduled metric, grouped by *account_id*
- A daily scheduled metric, grouped by *merchant_id*
- A weekly scheduled metric, grouped by *account_id, merchant_id*
- A weekly scheduled metric grouped by *terminal_id, account_id, merchant_id*

The daily SMJ is not partitionable. In fact, neither of its scheduled metrics are partitionable because they do not contain all the workflow partition keys. The weekly SMJ, instead, is partitionable.

Non-partitionable plans are handled the same way as when partitioning is disabled, i.e., an SMJ is registered for each unique metric schedule. On partitionable plans there is also the same grouping by metric schedule but, rather than registering a single entry per schedule, the following entries are registered for each schedule:

- An entry for all partitions, that allows processing all partitions on demand in a single execution. In this case, all partitions are processed sequentially in a single job execution.
- An entry per partition, that allows processing a single partition on demand.

To improve the performance of non-partitionable SMJs, they execute in a simulated partitionable mode. An SMJ can still be split into several jobs, one per available and configured partition, even though the scheduled metrics are not all partitionable.

Although it is still partitioned, this mode causes each split/partitioned job to read all events from the [EventStorage](#), for the time range covered. This is caused due to the fact that the **Event Storage** does not have a corresponding partition for non-partitionable metrics. Natively partitionable SMJs end up being more efficient since they need only read the corresponding partition from the **Event Storage**.

Contrarily to regular SMJs, there is no one-to-one relation between registered jobs and scheduled job executions - see the scheduled job execution section below.

Scheduled Job Execution

With partitioned SMJs, multiple partitions are to be processed according to the metric schedule, with each partition being processed by a separate job execution. Pulse provides a flexible way to specify when these executions are to be launched, allowing two behaviors:

- All partitions on the scheduled time.
- Dividing the partitions into sets (partition buckets) and distributing bucket processing by the schedule period. This allows distributing processing over time rather than having all partitions processed at the same instant; for example, on a weekly metric schedule, create daily buckets so the processing is distributed on a daily basis.

Regardless of the way SMJs are scheduled, the level of parallelism on which partition SMJs are executed depends only on how the Job Manager is configured (`externalSlots` property), i.e., it can go from strictly sequential to all in parallel.

All Partitions at the Scheduled Time

This is the default behavior and doesn't require any configuration. A job execution is created for each partition at the scheduled time, and the job manager will execute each as soon as possible taking into consideration the `externalSlots`

property and running job executions.

Distributing SMJ Processing with Partition Buckets

In addition, to reduce memory consumption, SMJ partitioning can also be used to distribute processing over time. For example, if we have a weekly schedule and the job total execution time is too long (e.g., if it is supposed to take place during the night but finishes inside business hours), it is preferable to distribute the partitions by each day of the week and thus have a shorter SMJ execution times, although more frequent.

This can be achieved using partition buckets, that is based on the following:

- The metric schedule interval is divided into sub-intervals, with SMJ execution being triggered on each sub-interval. Each SMJ execution processes a subset of the partitions, named partition bucket.
- The number of buckets in each metric schedule interval is defined indirectly by assigning a bucket schedule to the metric schedule. On the previous example, a daily bucket schedule would be assigned to a weekly metric schedule; the metric schedule period (7 days) has 7 bucket schedule periods (1 day each), thus we have 7 buckets.
- Partitions are distributed in a uniform way by buckets. For example, if we had 8 partitions and 7 buckets we would have 6 buckets with one partition and one bucket with two partitions.
- SMJs are launched on the bucket schedule, and each launch processes the partitions of launch time bucket. The bucket correspondent to each time is calculated from the metric and bucket schedules, by calculating the number of bucket schedule intervals that took place between the previous metric schedule time and the current time.

Thus, in order to enable partition buckets, it is only necessary to configure a bucket schedule for each metric schedule where buckets are to be used. This way we define both the number of buckets inside the metric schedule interval and the time when each bucket is to be processed.

Learn More

Check the [Workflow concept page](#) for all the details on Workflows.